Referencje i kopie obiektów

Referencje do obiektów

Przypisanie nie tworzy kopii obiektu. Ten sam mechanizm, który obserwowaliśmy dla typów wbudowanych, działa także dla obiektów naszych klas.

In [1]:
```
class Osoba:
    def __init__(self, imie, nazwisko, rok_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.rok_urodzenia = rok_urodzenia
```

Wypróbujmy podstawienie:

In [2]:
```
osoba_1 = Osoba("Jan", "Kowalski", 2000)
osoba_2 = osoba_1
```

i zmianę wartości pola:

In [3]:
```
osoba_2.rok_urodzenia = 2010
```

Teraz sprawdźmy wartość pola za pomocą innej zmiennej niż użyliśmy do zmiany wartości:

In [4]:
```
osoba_1.rok_urodzenia
```

Out[4]:
```
2010
```

Mamy teraz tylko jeden obiekt i dwie referencje do niego.

Sprawdź:

- Wizualizacja działania referencji do obiektów na stronie Python Tutor

Referencje jako pola

To samo dotyczy obiektów, które same są polami innych obiektów.

Zdefiniujmy klasę do reprezentowania daty:

In [5]:
```
class Data:
    def __init__(self, rok, miesiac, dzien):
        self.rok = rok
        self.miesiac = miesiac
        self.dzien = dzien
```

```
    def __repr__(self):
        return f"Data({self.rok!r}, {self.miesiac!r}, {self.dzien!r})"
```

Następnie poprawmy definicję klasy `Osoba` tak, żeby przechowywała całą datę urodzenia w polu `data_urodzenia` jako obiekt klasy `Data`.

In [6]:
```
class Osoba:
    def __init__(self, imie, nazwisko, data_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.data_urodzenia = data_urodzenia

    def __repr__(self):
        return (
            f"Osoba({self.imie!r}, {self.nazwisko!r}, "
            f"{self.data_urodzenia!r})"
        )
```

Tworzymy obiekt reprezentujący dzień 10 maja 2010 roku:

In [7]:
```
data = Data(2010, 5, 10)
```

Tworzymy dwa obiekty reprezentujące osoby urodzone tego dnia:

In [8]:
```
jan = Osoba("Jan", "Kowalski", data)
anna = Osoba("Anna", "Nowak", data)
```

Zmieniamy dzień urodzenia jednej z osób:

In [9]:
```
anna.data_urodzenia.dzien += 10
```

Sprawdzamy dzień urodzenia drugiej osoby:

In [10]:
```
jan.data_urodzenia.dzien
```

Out[10]:
```
20
```

Widzimy, że data została zmieniona. Spowodowane jest to tym, że osoby dzielą wspólny obiekt `Data`.

Zmiana widoczna jest też jeżeli użyjemy zmiennej `data`:

In [11]:
```
data
```

Out[11]:
```
Data(2010, 5, 20)
```

Sprawdź:

- Wizualizacja współdzielenia pól na stronie Python Tutor

Kopie obiektów

Jeżeli potrzebujemy kopii obiektu, to możemy ją zrobić tworząc nowy obiekt z identycznymi wartościami pól, np.

In [12]:
```
data = Data(2010, 5, 10)
kopia_daty = Data(data.rok, data.miesiac, data.dzien)
```

Teraz są to dwa niezależne obiekty:

In [13]:
```
data.dzien += 10
```

Sprawdźmy:

In [14]:
```
kopia_daty.dzien
```

Out[14]:
```
10
```

Sprawdź:

- Wizualizacja kopii obiektu na stronie Python Tutor

Możemy podobnie zrobić w konstruktorze klasy `Osoba` żeby uniknąć takiego powiązania obiektów wspólnymi danymi:

In [15]:
```
class Osoba:
    def __init__(self, imie, nazwisko, data_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.data_urodzenia = Data(
            data_urodzenia.rok,
            data_urodzenia.miesiac,
            data_urodzenia.dzien,
        )

    def __repr__(self):
        return (
            f"Osoba({self.imie!r}, {self.nazwisko!r}, "
            f"{self.data_urodzenia!r})"
        )
```

Sprawdźmy teraz:

In [16]:
```
data = Data(2010, 5, 10)
jan = Osoba("Jan", "Kowalski", data)
anna = Osoba("Anna", "Nowak", data)

anna.data_urodzenia.miesiac = 2
jan.data_urodzenia.miesiac = 7
```

In [17]:
```
anna.data_urodzenia
```

Out[17]:
```
Data(2010, 2, 10)
```

In [18]:
```
jan.data_urodzenia
```

Out[18]:
```
Data(2010, 7, 10)
```

In [19]:
```
data
```

Out[19]:
```
Data(2010, 5, 10)
```

Widać, że tym razem mamy trzy osobne obiekty typu `Data`.

Sprawdź:

- Wizualizacja tworzenia kopii obiektu w konstruktorze na stronie Python Tutor.

Moduł `copy`

Do kopiowania obiektów możemy też użyć funkcji z modułu `copy`:

- płytka kopia: copy.copy
- głęboka kopia: copy.deepcopy

Poniższe przykłady pokazują na czym polega płytka i głęboka kopia:

In [20]:
```
import copy
```

Samo przypisanie nie tworzy nowego obiektu:

In [21]:
```
data = Data(2010, 5, 10)
osoba = Osoba("Jan", "Kowalski", data)
referencja = osoba
```

In [22]:
```
referencja is osoba
```

Out[22]:
```
True
```

Płytka kopia

In [23]:
```
data = Data(2010, 5, 10)
osoba = Osoba("Jan", "Kowalski", data)
plytka_kopia = copy.copy(osoba)
```

Powstaje nowy obiekt `Osoba`, którego pola wskazują te same obiekty co pola oryginału. W szczególności pole `data_urodzenia` wskazuje tę samą datę.

Przypisanie nowej wartości do pola kopii zmienia tylko to pole kopii:

In [24]:
```
plytka_kopia.imie = "Piotr"
```

Sprawdźmy:

In [25]:
```
osoba.imie
```

Out[25]:
```
'Jan'
```

Teraz dla pola zagnieżdżonego obiektu:

In [26]:
```
plytka_kopia.data_urodzenia.dzien = 15
```

Sprawdźmy:

In [27]:
```
osoba.data_urodzenia.dzien
```

Out[27]:
```
15
```

Sprawdź:

- Wizualizacja tworzenia płytkiej kopii obiektu w konstruktorze na stronie Python Tutor

Głęboka kopia

In [28]:
```
data = Data(2010, 5, 10)
osoba = Osoba("Jan", "Kowalski", data)
gleboka_kopia = copy.deepcopy(osoba)
```

In [29]:
```
gleboka_kopia.imie = "Piotr"
```

Sprawdźmy, że imię w oryginale się nie zmieniło:

In [30]:
```
osoba.imie
```

Out[30]:
```
'Jan'
```

Teraz dla pola zagnieżdżonego obiektu:

In [31]:
```
gleboka_kopia.data_urodzenia.dzien = 15
```

Sprawdźmy:

In [32]:
```
osoba.data_urodzenia.dzien
```

Out[32]:
```
10
```

Powstał nowy obiekt `Osoba` z osobną kopią daty urodzenia. Zmiana tej daty nie wpływa na datę w oryginale.

Nie oznacza to, że `deepcopy` zawsze tworzy kopię każdego obiektu. Obiekty niemutowalne, np. napisy, mogą pozostać wspólne — nie przeszkadza to w niezależnym zmienianiu danych obu osób.

Sprawdź:

- Wizualizacja tworzenia głębokiej kopii obiektu w konstruktorze na stronie Python Tutor

Kopie innych struktur danych

Podobnie działa to np. dla list (oraz innych typów złożonych takich jak słowniki czy krotki). Poniżej tworzymy listę, której ostatnim elementem jest również lista.

Tworzymy listę i jej płytką kopię:

In [33]:
```
lista = [1, 2, [3, 4]]
kopia_listy = copy.copy(lista)
```

In [34]:
```
kopia_listy
```

Out[34]:
```
[1, 2, [3, 4]]
```

Lista została skopiowana, ale ostatni element (lista `[3, 4]`) jest wspólny:

In [35]:
```
kopia_listy[0] = -1
kopia_listy[2][0] = -3
```

Sprawdźmy:

In [36]:
```
kopia_listy
```

Out[36]:
```
[-1, 2, [-3, 4]]
```

In [37]:
```
lista
```

Out[37]:
```
[1, 2, [-3, 4]]
```

Sprawdź:

- Wizualizacja płytkiej kopii listy na stronie Python Tutor

Teraz tworzymy głęboką kopię:

In [38]:
```
lista = [1, 2, [3, 4]]
gleboka_kopia_listy = copy.deepcopy(lista)
```

Powstała nowa lista, a jej zagnieżdżona lista również została skopiowana. Możemy zmieniać jej elementy niezależnie od oryginału.

In [39]:
```
gleboka_kopia_listy[0] = -1
gleboka_kopia_listy[2][0] = -3
```

Sprawdźmy:

In [40]:
```
gleboka_kopia_listy
```

Out[40]:
```
[-1, 2, [-3, 4]]
```

In [41]:
```
lista
```

Out[41]:
```
[1, 2, [3, 4]]
```

Sprawdź:

- Wizualizacja głębokiej kopii listy na stronie Python Tutor

Definicja klasy `Osoba` używająca modułu `copy` mogłaby wyglądać tak:

In [42]:
```
class Osoba:
    def __init__(self, imie, nazwisko, data_urodzenia):
        self.imie = imie
        self.nazwisko = nazwisko
        self.data_urodzenia = copy.copy(data_urodzenia)
```

Kopiujemy wyłącznie obiekt daty. Jego pola wskazują liczby całkowite, które są niemutowalne, więc wystarczy tutaj płytka kopia.